Part 1: Mathematical Theory

T0: General Notation

- d ... dimension of the nonlinear problem
- $\Omega \in \mathbb{R}^d$ ... domain of interest
- $F: \Omega \to \mathbb{R}^d$ ... function whose root we want to find
- x^* ... exact root, $F(x^*) = 0$
- $x^{(0)}$ ... initial approx. / $x^{(k)}$ ... approx after k iterations
- φ ... fixed point iteration map
- $DF(x)$ / $D\varphi(x)$ ... jacobians

T1. What is Nullstellensuche?

We now want to solve $F(x) = 0$

In one dimension a root is just a point where the graph of F intersects with the horizontal axis.

In two dimensions we require:

$$F(x,y) = \begin{pmatrix} F_1(x,y) \\ F_2(x,y) \end{pmatrix} = 0$$

$\Rightarrow F_1(x,y) = 0$ and $F_2(x,y) = 0$ must hold.

General Strategy

Rather than calculating x^* directly we construct a sequence:

$$x^{(0)}, x^{(1)}, x^{(2)}, \dots$$

and hope that

$$x^{(k)} \to x^*$$

→ We must observe whether the alg converges, and if it does how fast it converges!

T2. Iterative Methods

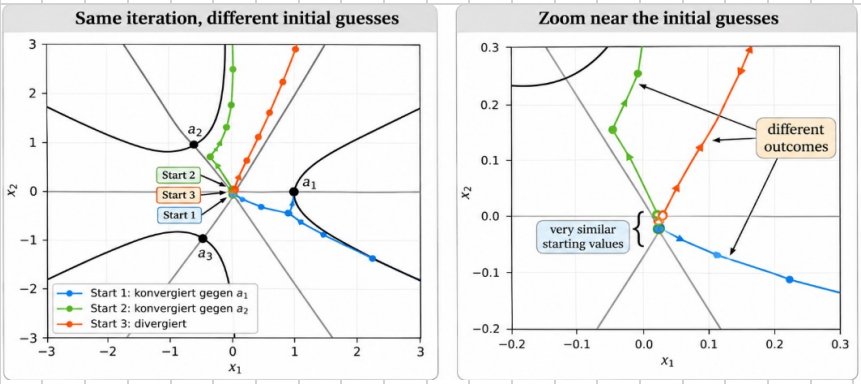
These methods generate the approximation sequence.

For one step we write $x^{(k+1)} = \varphi(x^{(k)})$ — current step!

Def. 6.1.4

An iterative method for $F(x) = 0$ converges if $x^{(k)} \to x^*$ and $F(x^*) = 0$

Initial guess is important, it is part of the problem!



Extra: We will measure the distances between vectors by norms. For vetors in $\mathbb{R}^d$ common norms are;

$$\|x\|_1 = \sum_{i=1}^{d} |x_i| \qquad \|x\|_2 = \sqrt{\sum_{i=1}^{d} |x_i|^2} \qquad \|x\|_\infty = \max_i |x_i|$$

Def 6.1.9 Equivalent norms

c, C ... constants $\to$ $c\|x\|_a \le \|x\|_b \le C\|x\|_a$ for vectors x.

↳ All norms on a finite-dimensional vector space are equivalent, e.g. if a sequence converges in the Euclidean norm, it also converges in inf. norm. But the error values can differ

Example 6.1.11

For $x \in \mathbb{R}^d$,

$$\|x\|_2 \le \|x\|_1 \le \sqrt{d}\,\|x\|_2$$

$$\|x\|_\infty \le \|x\|_2 \le \sqrt{d}\,\|x\|_\infty$$

$$\|x\|_\infty \le \|x\|_1 \le d\,\|x\|_\infty$$

Error and residual

$e^{(k)} = x^{(k)} - x^*$ and its magnitude is $\varepsilon_k = \|e^{(k)}\|$

Residual ... $r^{(k)} = F(x^{(k)})$

→ Be careful these are different

- Error $e^{(k)}$ measures how far are we from root!
- Residual $r^{(k)}$ measures how far are we from 0!

Linear Convergence

A sequence converges linearly if there exists $L < 1$ s.t, for all sufficiently large k

$$\|x^{(k+1)} - x^*\| \leq L \|x^{(k)} - x^*\|$$

convergence factor

→ Repeadetly apply this:

$$\|x^{(k+1)} - x^*\| \leq L\|x^{(k)} - x^*\| \leq L.L.\|x^{(k-1)} - x^*\| \leq \dots \leq L^k \|x^{(0)} - x^*\|$$

$\Leftrightarrow \varepsilon_k \leq L^k \varepsilon_0$, thus because $0 < L < 1$ $\varepsilon_k \to 0$

General Convergence Order

An iteration has order p if an estimate of the form

$$\varepsilon_{k+1} \leq C \varepsilon_k^p$$

holds with $C < 1$.

→ $p = 1$ .. linear $p > 1$ ... superlinear $p = 2$ ... quadratic

Estimating Convergence Order

⇒ $\varepsilon_{k+1} = C\varepsilon_k^p$ ~> Take log: $\log \varepsilon_{k+1} = \log C + p \log \varepsilon_k$. Do same for k

⇒ Then $\log \varepsilon_{k+1} - \log \varepsilon_k = p(\log \varepsilon_k - \log \varepsilon_{k-1})$

⇒ $p \approx \dfrac{\log \varepsilon_{k+1} - \log \varepsilon_k}{\log \varepsilon_k - \log \varepsilon_{k-1}}$

T3. Stopping Criterion

As a computer cannot perform infinitely many iterations, we must decide when the current approx. is sufficiently accurate.

Well, the ideal one is $\|x^{(k)} - x^*\| < \tau$ but x^* is unknown :(

Common Stopping Strategies

- Fixed iteration count
- Residual criterion: $\|F(x^{(k)})\| = \|r^{(k)}\| < \tau$
  small residual $\not\Rightarrow$ small error!
- Successive-iterate criterion: $\|x^{(k+1)} - x^{(k)}\| < \tau$
- **Error estimate:** → we estimate the unknown error
  - → a priori criterion: estimates the necessary number of iterations
  - → a posteriori criterion:

E2

Assume $\|x^{(k+1)} - x^*\| \leq L \|x^{(k)} - x^*\| \quad L < 1$

$$L\|x^{(k)} - x^*\| = L\|x^{(k)} - x^{(k+1)} + x^{(k+1)} - x^*\|$$

$$\overset{\Delta\text{-ineq.}}{\leq} L\|x^{(k+1)} - x^{(k)}\| + L\|x^{(k+1)} - x^*\| \quad (A)$$

→ Using (A) we get:

$$(1-L)\|x^{(k+1)} - x^*\| \leq L\|x^{(k+1)} - x^{(k)}\|$$

$$\Leftrightarrow \quad \|x^{(k+1)} - x^*\| \leq \frac{L}{1-L}\|x^{(k+1)} - x^{(k)}\| \quad (\star)$$

($\|x^{(k+1)} - x^{(k)}\|$: known values)

Use $(\star)$ as errest_{k+1} and stop when $\text{errest}_{k+1} < \tau$

I4 Fixed-point iteration

Def 6.3.1 – Consistence

A fixed-point it. is consistent with $F(x) = 0$ if

$$F(x) = 0 \iff \varphi(x) = x$$

so this mapping φ doesnt change iff. $F(x) = 0$ at that point, but for $x^{(k)}$ it gives $\varphi(x^{(k)}) = x^{(k+1)}$!

If we find where $\varphi(x) = x$ holds then we find root of F!

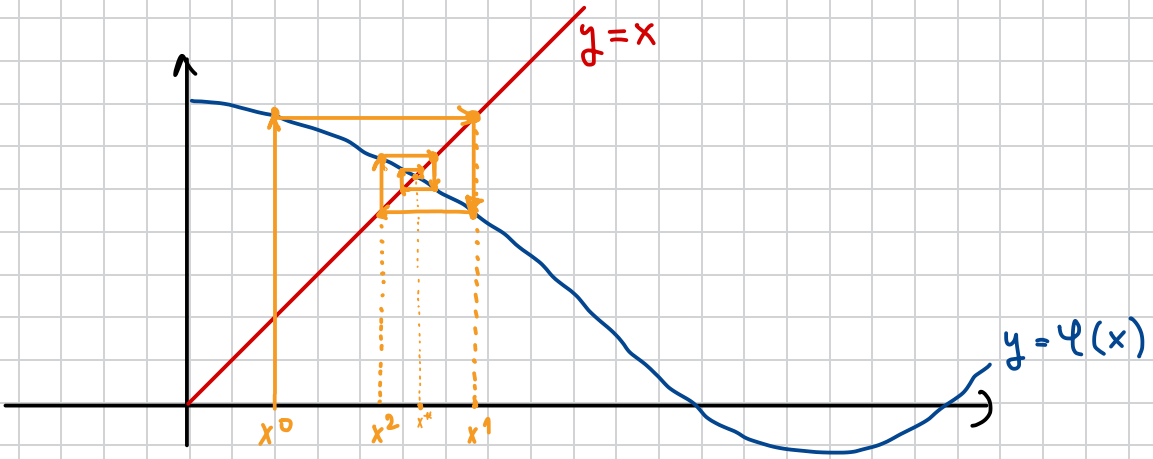
→ $y = \varphi(x)$ and $y = x$ ... where these two meet is where $\varphi(x) = x$ holds!

To visualize fixed-point iteration:

Start from $x^{(0)}$ → Move vertically to $y = \varphi(x)$

Then move horizontally to the diagonal $y = x$ → New horizontal coord is $x^{(1)}$

repeat!

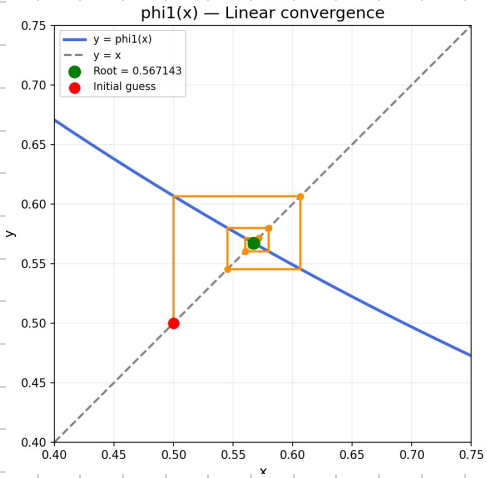


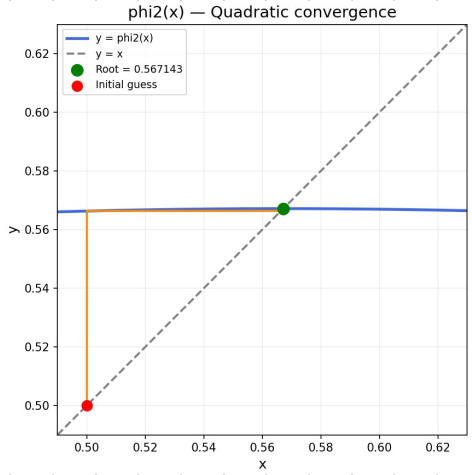
Example 6.3.2

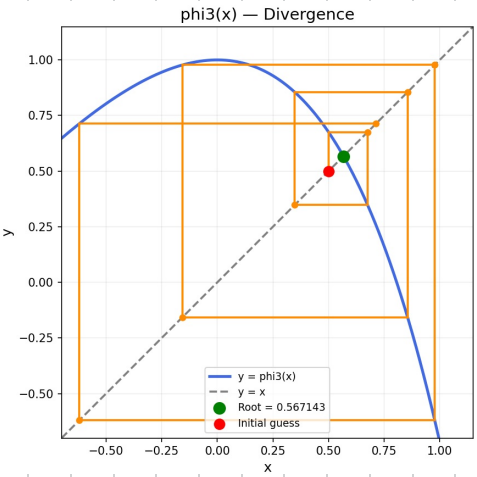
$F(x) = xe^x - 1$ we introduce fixed-point maps:

$$\varphi_1(x) = e^{-x}, \quad \varphi_2(x) = \frac{1+x}{1+e^x}, \quad \varphi_3(x) = x + 1 - xe^x$$

Observe $\varphi_i(x) = x$ ... each has same fixed point







Why does derivative control convergence?

Suppose $\varphi(x^*) = x^*$ and $e^{(k+1)} = \varphi(x^{(k)}) - \varphi(x^*)$

$\Rightarrow$ Taylor expansion gives $\varphi(x^{(k)}) = \varphi(x^*) + \varphi(x^*)'(x^{(k+1)} - x^*) + \frac{\varphi(x^*)''}{2!}(x^{(k+1)} - x^*)^2 + \dots$

$$= \varphi(x^*) + \varphi(x^*)' e^{(k)} + \frac{\varphi(x^*)''}{2}(e^{(k)})^2 + \dots$$

$$\Rightarrow e^{(k+1)} = \varphi(x^{(k)}) - \varphi(x^*) = \varphi(x^*)' e^{(k)} + \frac{\varphi(x^*)''}{2}(e^{(k)})^2 = \varphi(x^*)' e^{(k)} + O((e^{(k)})^2)$$

$\Rightarrow$ For sufficiently small errors we have, if $\varphi(x^*)' \neq 0$:

$$e^{(k+1)} \approx \varphi(x^*)' e^{(k)}$$

Derivative at x^*	Behaviour
$0 < \varphi'(x^*) < 1$	Converges
$-1 < \varphi'(x^*) < 0$	Converges
$\varphi'(x^*) = 0$	Potentially higher order convergence
$\varphi'(x^*) > 1$	Diverges
$\varphi'(x^*) < -1$	Diverges
$\lvert\varphi'(x^*)\rvert = 1$	Inconclusive

→ Do it for Ex 6.3.2!

Def. 6.3.5

A map φ is a **contraction** if there exists $0 \le L < 1$ s.t.

$$\|\varphi(x) - \varphi(y)\| \le L \|x - y\| \quad \forall x, y$$

↳ map φ makes any two points closer together!

Remark 6.3.6

If x^* is a fixed point then

$$\|x^{(k+1)} - x^*\| = \|\varphi(x^{(k)}) - \varphi(x^*)\| \le L \|x^{(k)} - x^*\|$$

(↖ Def 6.3.5)

⇒ Contraction property directly implies linear convergence!

Banach's Fixed-Point Theorem

Let $\Omega \in \mathbb{R}^d$ be nonempty and closed, and $\varphi : \Omega \to \Omega$ is a contraction.

Then:
1. A fixed point $x^* \in \Omega$ exists and it is unique
2. The iter. converges to x^* from every initial value

Th. Intervalhalbierungsverfahren ($d = 1$)

Let F be continuous, we still want roots.

Suppose for some a, b we have $F(a) < 0$ and $F(b) > 0$

The intermediate value theorem guarantees that there is at least one point $x^* \in (a, b)$ satisfying

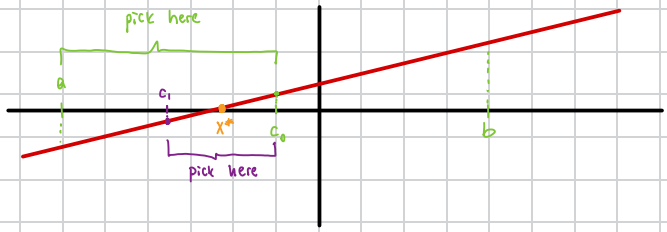
$$F(x^*) = 0$$

Bisection

This method repeatedly reduces the interval while keeping the root inside.

Idea: Take $c = \frac{a+b}{2}$ now we have $[a, c]$ and $[c, b]$, evaluate $F(c)$.

If $F(a)F(c) < 0$ sign change occurs in $[a, c]$, else if $F(c) \neq 0$ then in $[c, b]$



Bisection converges because at every iteration:

$$b_{k+1} - a_{k+1} = \frac{1}{2}(b_k - a_k)$$

$$\Rightarrow b_k - a_k = \frac{b_0 - a_0}{2^k} \quad \text{follows}$$

Also if our approx. is $c_k = \frac{a_k - b_k}{2}$ the distance from the midpoint to any root inside the interval is at most half the interval length.

$$\Rightarrow |c_k - x^*| \leq \frac{b_k - a_k}{2}$$

$$\Rightarrow |c_k - x^*| \leq \frac{b_0 - a_0}{2^{k+1}} \quad \text{... error bound}$$

How many iterations are necessary?

Suppose we want $|c_k - x^*| \leq \tau$, it suffices to require $\frac{b_0 - a_0}{2^{k+1}} \leq \tau$

$\Rightarrow$ Rearrange to get $k+1 \geq \log_2\left(\frac{b_0 - a_0}{\tau}\right)$

We can therefore determine a sufficient number of bisection steps before executing the algorithm.

$\Rightarrow$ Example of an a priori error estimate

1b. Newton Method in one dimension

Idea is to replace the nonlinear function locally by its tangent line.

If F is sufficiently smooth $F(x+h) = F(x) + F'(x)h + O(h^2)$ and for small h linear term dominates:

$$F(x+h) \approx F(x) + F'(x)h$$

At the current approx. $x^{(k)}$ we write

$$\tilde{F}^{(k)}(x) = F(x^{(k)}) + F'(x^{(k)})(x - x^{(k)})$$

This is the tangent line, we find the root of this instead.

Derivation of Newton's formula

Set $\tilde{F}^{(k)}(x^{(k+1)}) = 0$, therefore $F(x^{(k)}) + F'(x^{(k)})(x^{(k+1)} - x^{(k)}) = 0$

$\Rightarrow F'(x^{(k)})(x^{(k+1)} - x^{(k)}) = -F(x^{(k)})$ and assume $F'(x^{(k)}) \neq 0$

$\Leftrightarrow x^{(k+1)} - x^{(k)} = -\frac{F(x^{(k)})}{F'(x^{(k)})}$ $\Leftrightarrow$ Thus $x^{(k+1)} = x^{(k)} - \frac{F(x^{(k)})}{F'(x^{(k)})}$

and $s^{(k)} = \frac{F(x^{(k)})}{F'(x^{(k)})}$ is the Newton correction

- Newton is locally (→ there are some failure cases) quadratically convergent.
- It can also be written as an iterative map!
- **Multiple Roots**

Suppose $F(x) = (x-a)^m$ $m \geq 2$. The root is a but $F'(a) = 0$

$\Rightarrow F'(x) = m(x-a)^{m-1}$

$\Rightarrow$ Newton update becomes: $x^{(k+1)} = x^{(k)} - \frac{(x^{(k)} - a)^m}{m(x^{(k)} - a)^{m-1}}$

$$= x^{(k)} - \frac{x^{(k)} - a}{m}$$

$\Rightarrow$ Subtract a ($x^{(k)} \to x^*$) $x^{(k+1)} - a = x^{(k)} - \frac{x^{(k)} - a}{m} - a$ ($a \to x^*$)

$\Leftrightarrow e^{(k+1)} = \left(1 - \frac{1}{m}\right) e^{(k)}$ ... Newton slows down with increasing m. ($x^{(k+1)} - x^* = e^{(k+1)}$)

- In many problems the derivative is unavailable, we can approx. it using finite differences.

→ $D_h F(x) = \frac{F(x+h) - F(x)}{h}$ or $D_h^{\text{central}} F(x) = \frac{F(x+h) - F(x-h)}{2h}$

T7. Secant Method

Secant line approx. through the current point is

$$\tilde{F}^{(k)}(x) = F(x^{(k)}) + q^{(k)}(x - x^{(k)})$$

Suppose $F(x^{(k)}) + q^{(k)}(x - x^{(k)}) = 0$

⇒ Rearrange to get: $x^{(k+1)} = x^{(k)} - \frac{F(x^{(k)})}{q^{(k)}}$

⇒ Substitute the secant slope:

$$x^{(k+1)} = x^{(k)} - \frac{x^{(k)} - x^{(k-1)}}{F(x^{(k)}) - F(x^{(k-1)})} F(x^{(k)})$$

- Two starting values $x^{(0)}$ and $x^{(1)}$ are needed
- Can fail when $F(x^{(k)}) - F(x^{(k-1)})$ is 0, too small or too large.
- Order around 1.618 (no need to calculate it)

T8. Mathematical interlude - The Jacobian

$$F(x,y) = \begin{pmatrix} F_1(x,y) \\ F_2(x,y) \end{pmatrix} \Rightarrow DF(x,y) = \begin{pmatrix} \partial_x F_1 & \partial_y F_1 \\ \partial_x F_2 & \partial_y F_2 \end{pmatrix} \rightarrow \nabla F_1(x,y)$$

⇒ In 1D we did $f(x+h) \approx f(x) + f'(x)h$ (h scalar)

⇒ Jacobian generalizes this: $F(x+h) \approx F(x) + DF(x)h$ (h vector)

T9. Newton-Method in d dimensions

Same problem outline $F(x) = 0$ but $F: \mathbb{R}^d \to \mathbb{R}^d$

$$\Rightarrow x^{(k+1)} = x^{(k)} - \frac{F(x^{(k)})}{DF(x^{(k)})} \quad \text{and} \quad s^{(k)} = DF(x^{(k)})^{-1} F(x^{(k)})$$

$$\Rightarrow DF(x^{(k)})\, s^{(k)} = F(x^{(k)}) \quad \dots \quad s = \text{np.linalg.solve}(DFx, Fx)$$

Workflow: Evaluate $x^{(i)}$ to get $F(x^{(i)})$. Compute $DF(x^{(i)})$. Solve for $s^{(i)}$. Update to get $x^{(i+1)}$. Repeat

Computing derivatives using SymPy

```
from sympy import symbols, diff
x, y = symbols("x", "y")
g = x**2 + 2*y**2
dg_dx = diff(g, x)
dg_dy = diff(g, y)
```
} symbolic expressions

```
dg_dx_func = lambdify((x, y), dg_dx, "numpy")
dg_dy_func = lambdify((x, y), dg_dy, "numpy")
```
⇝
```
def dg_dx_func(x, y):
    return 2*x
```

- Stopping criterion using Newton correction near a solution/root:

$$\|s^{(k)}\| = \|DF(x^{(k)})^{-1} F(x^{(k)})\| \leq \tau$$

⇒ Or simplified:

$$\|DF(x^{(k-1)})^{-1} F(x^{(k)})\| \leq \tau \quad \rightarrow \text{Stop when holds}$$

- **Why is multidimensional Newton expensive?**

For a dense $d \times d$ Jacobian, an LU factorization costs $O(d^3)$ arithmetic operations. After that solving the system is $O(d^2)$.

→ If d is large we have a problem

Simplified Newton Method

Ordinary Newton recomputes the Jacobian: $DF(x^{(0)}), DF(x^{(1)}), \ldots$

→ Simplified Newton keeps a **Jacobian fixed for several steps**

Freeze it at $\bar{x}$ → $J = DF(\bar{x})$. Iteration becomes $J s^{(k)} = F(x^{(k)})$, $x^{(k+1)} = x^{(k)} - s^{(k)}$

! Same LU factorization can be reused

→ Cheaper iterations, but slower convergence

T10. Damped Newton Method

Example: $F(x) = \arctan(x)$, root is $x^* = 0$ and $F'(x) = \frac{1}{1+x^2}$

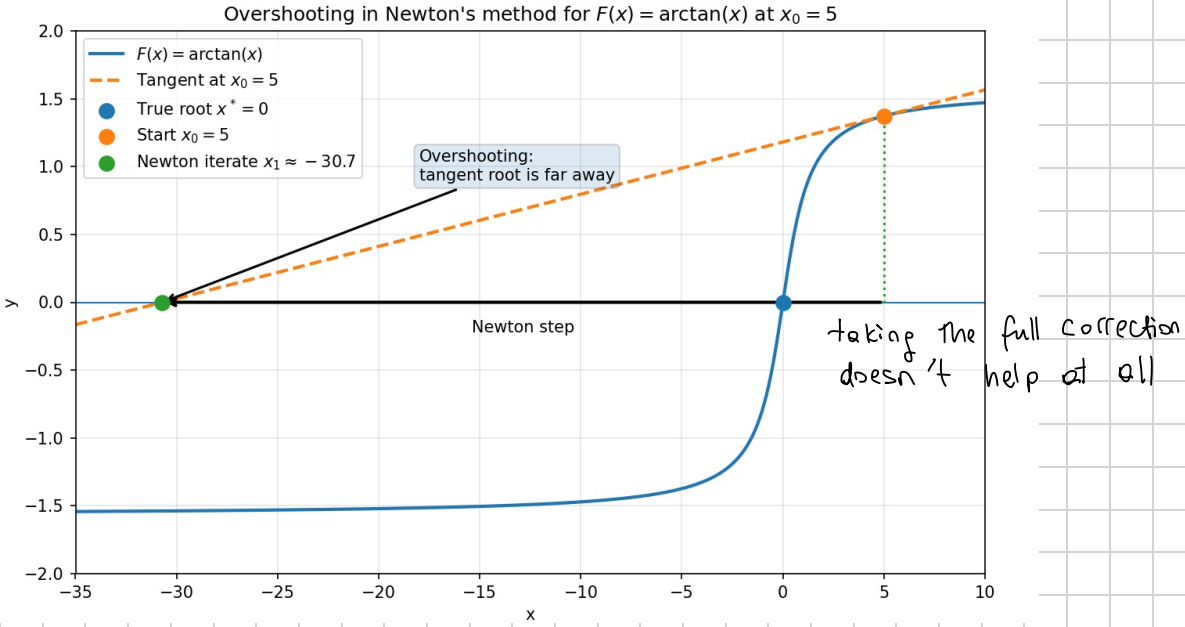
$\Rightarrow s(x) = \frac{F(x)}{F'(x)} = (1+x^2)\arctan(x)$

$\Rightarrow$ If $x = 5$, then $F(5) \approx 1.3734$ and $F'(5) = \frac{1}{26}$

$\Rightarrow s(5) = 26 \cdot 1.3734 \approx 35.7$

$\Rightarrow$ Newton proposes $x_{new} \approx 5 - 35.7 = -30.7$

$\Rightarrow$ The step overshoots



Damping Parameter

Instead of taking the full correction, choose $0 < \lambda^{(k)} < 1$.

$$\Rightarrow x^{(k+1)} = x^{(k)} - \lambda^{(k)} s^{(k)}$$

How to choose $\lambda^{(k)}$?

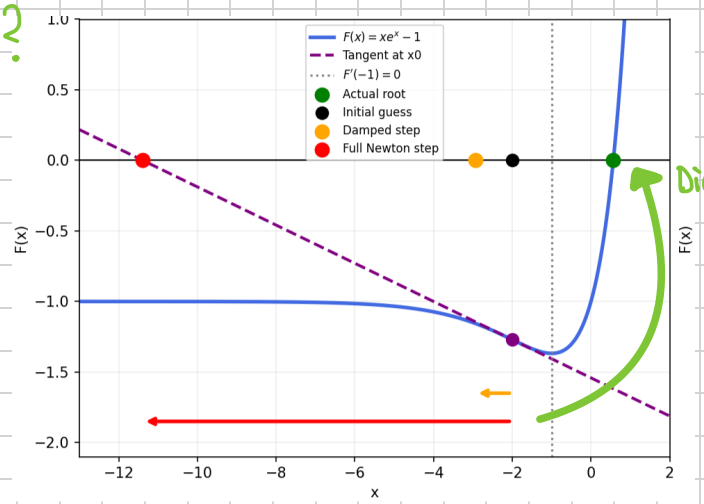
Script uses a heuristic on the natural monotonicity condition.

- Compute $s^{(k)} = DF(x^{(k)})^{-1} F(x^{(k)})$
- For a candidate λ define $x_{trial} = x^{(k)} - \lambda s^{(k)}$
- $s_{trial} = DF(x^{(k)})^{-1} F(x_{trial})$ reusing the Jacobian
- $\|s_{trial}\|_2 \leq \left(1 - \frac{\lambda}{2}\right) \|s^{(k)}\|_2$

- **Backtracking**: Start with $\lambda = 1$, if not satisfied reduce λ and test!

Damping is applied only at needed steps, so original convergence of Newton Method could be recovered.

Where does this fail?



Direction is wrong!

T11. Quasi-Newton Method

The idea is to approximate the Jacobian and avoid the costly part of Newton Method.

$$F'(x^{(k)}) \approx \frac{F(x^{(k)}) - F(x^{(k-1)})}{x^{(k)} - x^{(k-1)}} \quad \text{in 1D.}$$

Multidimensional

Call our approximation $J_k \approx \frac{\Delta F}{\Delta x} \Rightarrow J_k \Delta x = \Delta F$ (matrix; Now vectors)

$$\Rightarrow J_k \left(x^{(k)} - x^{(k-1)} \right) = F(x^{(k)}) - F(x^{(k-1)})$$

Broyden's principle

Suppose we have J_k and we want J_{k+1}.

$\Rightarrow \Delta x_k = x^{(k+1)} - x^{(k)}$ and $\Delta F_k = F(x^{(k+1)}) - F(x^{(k)})$

$\Rightarrow$ We want $J_{k+1} \Delta x_k = \Delta F_k$ assume $(J_k + U_k)\Delta x_k = \Delta F_k$

$\Rightarrow$ Rearrange $U_k \Delta x_k = \Delta F_k - J_k \Delta x_k \Rightarrow U_k \Delta x_k = v_k$

$\Rightarrow$ We need some U_k s.t. $U_k \Delta x_k = v_k$

$\Rightarrow$ Simple choice is $U_k = \frac{V_k \Delta x_k^T}{\Delta x_k^T \Delta x_k}$

$\Rightarrow$ Check $U_k s_k = \frac{V_k \Delta x_k^T \Delta x_k}{\Delta x_k^T \Delta x_k} = V_k$

$\Rightarrow$ Therefore $J_{k+1} = J_k + \frac{(\Delta F_k - J_k \Delta x_k)\Delta x_k^T}{\Delta x_k^T \Delta x_k}$ → Broyden Update

- Initial approx. $J_0 = DF(x^{(0)})$!

Code Expert: Skalare nicht-linear Gleichung

We are asked to find the positive solution for

$$e^{2x} - \sin(x) = 2$$

Coding Tips

- you can use sympy for derivation, don't forget lambdify!
- atol, rtol = 1e-7 can be used
- give meaningful initial guesses
- for f(x) you have to implement $e^{2x} - \sin(x) - 2$

Code Expert: Nicht-lineares System

$$\begin{cases} e^{xy} + x^2 + y - \frac{6}{5} = 0 \\ x^2 + y^2 + x - \frac{11}{20} = 0 \end{cases}$$

, some starting values are also given

In newton you will need to keep a history of approximations → simple array and appending will do the job here.

Coding Tips

- use both maxiter and rtol * ... as stopping condition !
- How to compute the Jacobian using SymPy? Might need np.asarray(...)
- Relevant formula for estimating Conv. order is in the notes!

Code Expert: Ellipsoide

We are looking for the intersection point of all three ellipssiods.

Convert problem:

$$F(w) = \begin{pmatrix} F_1(w) \\ F_2(w) \\ F_3(w) \end{pmatrix} \quad \text{and} \quad F(w) = 0 \quad \text{with} \quad w = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

↑ first need to lambdify these as well

⇒ A multidimensional Newton will do the job here

Coding Tips

- Construct $DF(w)$ beforehand to give it as input to newton(...).
  ↳ will need lot of steps now!

$$DF(w) = \begin{bmatrix} \partial_x F_1 & \partial_y F_1 & \partial_z F_1 \\ \partial_x F_2 & \partial_y F_2 & \partial_z F_2 \\ \partial_x F_3 & \partial_y F_3 & \partial_z F_3 \end{bmatrix}$$

↑ diff and lambdify for all!

Code Expert: Broyden

Coding Tips

- Implement Newton but this time LU factorize the Jacobian and then solve the system!
- In simplified Newton don't recompute Jacobian reuse it.
- Refer to code 6.3.4 for Broyden!
- don't forget to change the line in aufgabe_b!

→ Code Examples